

جشنواره

«نایرا» مشغول یک بازی در یک جشنواره است که جایزه‌ی بزرگ آن سفر به «لاگونا کلرادو» است. این بازی شامل استفاده از سکه‌ها برای خرید کوپن است. خرید یک کوپن ممکن است منجر به دریافت سکه‌های اضافی شود. هدف این است که تا حد امکان کوپن‌های بیشتری خریداری کند.

او بازی را با A سکه شروع می‌کند. در کل N کوپن وجود دارد که از 0 تا $N - 1$ شماره‌گذاری شده‌اند. نایرا باید برای خرید کوپن i ($0 \leq i < N$)، $P[i]$ سکه بپردازد (و او باید قبل از خرید حداقل $P[i]$ سکه داشته باشد). او هر کوپن را حداکثر یک بار می‌تواند خریداری کند.

علاوه بر این، به هر کوپن i ($0 \leq i < N$) یک نوع اختصاص شده، که با $T[i]$ نشان داده می‌شود، که یک عدد صحیح بین 1 تا 4 (شامل هر دو) است. بعد از این که نایرا کوپن i را خریداری کرد، تعداد سکه‌های باقی‌مانده‌ی او در $T[i]$ ضرب می‌شود. به بیان دیگر، اگر او در مقطعی از بازی X سکه داشته باشد، و کوپن i را خریداری کند (که به $X \geq P[i]$ سکه نیاز دارد)، او پس از خرید، $(X - P[i]) \times T[i]$ سکه خواهد داشت.

شما باید مشخص کنید نایرا باید کدام کوپن‌ها را و به چه ترتیبی خریداری کند، که بتواند تعداد کوپن‌های خریداری شده را بیشینه کند. اگر بیش از یک توالی خرید وجود دارد که این امر را محقق می‌کند، می‌توانید هر یک از آن‌ها را گزارش کنید.

جزئیات پیاده‌سازی

شما باید تابع زیر را پیاده‌سازی کنید.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
std::vector<int> T)
```

- A : تعداد اولیه سکه‌های نایرا.
- P : آرایه‌ای به طول N که قیمت کوپن‌ها را مشخص می‌کند.
- T : آرایه‌ای به طول N که نوع کوپن‌ها را مشخص می‌کند.
- این تابع دقیقاً یک بار برای هر تست‌کیس فراخوانی می‌شود.

این تابع باید یک آرایه R برگرداند که خریده‌های نایرا را به شرح زیر مشخص می‌کند:

- طول R باید حداکثر تعداد کوپن‌هایی باشد که او می‌تواند خریداری کند.
- عناصر آرایه، اندیس کوپن‌هایی هستند که او باید به ترتیب زمانی خریداری کند. یعنی، او ابتدا کوپن $R[0]$ ، سپس کوپن $R[1]$ و به همین ترتیب خریداری می‌کند.
- تمام عناصر R باید متمایز باشند.

اگر هیچ کوپنی قابل خریداری نباشد، R باید یک آرایه خالی باشد.

محدودیت‌ها

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ برای هر i که $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ برای هر i که $0 \leq i < N$.

زیرمسئله‌ها

Subtask	Score	Additional Constraints
1	5	$T[i] = 1$ for each i such that $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ for each i such that $0 \leq i < N$.
3	12	$T[i] \leq 2$ for each i such that $0 \leq i < N$.
4	15	$N \leq 70$
5	27	نایرا می‌تواند تمام کوپن‌ها را خریداری کند (با یک ترتیب خاص).
6	16	$(A - P[i]) \times T[i] < A$ for each i such that $0 \leq i < N$.
7	18	بدون هیچ محدودیت اضافی.

مثال‌ها

مثال ۱

فراخوانی زیر را در نظر بگیرید.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

در ابتدا نایرا $A = 13$ سکه دارد. او می‌تواند 3 کوپن به ترتیب زیر خریداری کند:

تعداد سکه‌ها بعد از خرید	نوع کوپن	قیمت کوپن	کوپن خریده‌شده
$(13 - 8) \times 3 = 15$	3	8	2
$(15 - 14) \times 4 = 4$	4	14	3
$(4 - 4) \times 1 = 0$	1	4	0

در این مثال، نایرا نمی‌تواند بیشتر از 3 کوپن بخرد، و توالی خریدهای بالا تنها راهی است که او می‌تواند 3 کوپن بخرد. از این رو، تابع باید $[2, 3, 0]$ را برگرداند.

مثال ۲

فراخوانی زیر را در نظر بگیرید.

```
max_coupons(9, [6, 5], [2, 3])
```

در این مثال، نایرا می‌تواند هر دو کوپن را به هر ترتیبی خریداری کند. از این رو، این تابع باید یا $[0, 1]$ یا $[1, 0]$ را برگرداند.

مثال ۳

فراخوانی زیر را در نظر بگیرید.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

در این مثال، نایرا ۱ سکه دارد که برای خرید هیچ کوپنی کافی نیست. از این رو، این تابع باید $[]$ (یک آرایه خالی) را برگرداند.

گریدر نمونه

فرمت ورودی:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

فرمت خروجی:

```
S
R[0] R[1] ... R[S-1]
```

در این‌جا، S طول آرایه R است که توسط `max_coupons` برگردانده می‌شود.